Implementation of State Estimation for Bipedal Robot

Van Thanh Nguyen

Winter - 2026

1 StateEstimateBase Class

The `StateEstimateBase` class is an abstract base class used for state estimation in the legged robot framework. It provides common functionality for handling:

- Joint encoder measurements
- IMU measurements
- Contact state information
- Robot state storage
- Odometry publishing

The actual estimation algorithm is implemented in derived classes through the pure virtual function `update()`.

1.1 Class Purpose

The class maintains the robot state vector

$$x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix}, \tag{1.1}$$

where q denotes the generalized coordinates and $\dot{q}$ denotes the generalized velocities.

The state is stored in

```
vector_t rbdState_;
```

which is initialized as

```
rbdState_(vector_t::Zero(
    2 * info_.generalizedCoordinatesNum))
```

so that

$$\texttt{rbdState} = \begin{bmatrix} q \\ \dot{q} \end{bmatrix}. \tag{1.2}$$

1.2 Constructor

The constructor is defined as

```
StateEstimateBase(
    rclcpp::Node::SharedPtr node,
    PinocchioInterface pinocchioInterface,
    CentroidalModelInfo info,
    const PinocchioEndEffectorKinematics&
        eeKinematics);
```

Its responsibilities are:

1. Store the ROS node handle.

2. Store the Pinocchio interface.
3. Store robot model information.
4. Clone the end-effector kinematics object.
5. Allocate the state vector.
6. Create odometry and pose publishers.

1.3 State Vector Layout

Let

$$N = \texttt{info_.generalizedCoordinatesNum}. \tag{1.3}$$

The state vector layout is

Indices	Description
0–2	Base orientation (ZYX Euler angles)
3–5	Base position
6–	Joint positions
$N+0$ – $N+2$	Base angular velocity
$N+3$ – $N+5$	Base linear velocity
$N+6$ –	Joint velocities

Table 1: Layout of `rbdState`.

1.4 Joint State Update

The function

```
updateJointStates(
    const vector_t& jointPos,
    const vector_t& jointVel);
```

stores the encoder measurements into the state vector:

$$\texttt{rbdState}(6 : 6 + n_j - 1) = \texttt{jointPos}, \tag{1.4}$$

$$\texttt{rbdState}(N + 6 : N + 6 + n_j - 1) = \texttt{jointVel}, \tag{1.5}$$

where n_j is the number of actuated joints.

1.5 Contact State Update

The function

```
updateContact(contactFlag);
```

stores the current foot contact configuration.
This information is later used by state estimation algorithms such as:

- Linear Kalman Filters
- Contact-aided localization
- Base velocity estimation

1.6 Locomotion Mode

The function

`size_t getMode();`

converts the contact pattern into a locomotion mode identifier:

$$\texttt{mode} = \texttt{stanceLeg2ModeNumber(contactFlag)}. \tag{1.6}$$

Typical modes include:

- Left stance
- Right stance
- Double support
- Flight phase

1.7 IMU Update

The function

`updateImu(...)`

processes IMU measurements.
Its inputs include:

- Orientation quaternion
- Angular velocity
- Linear acceleration
- Covariance matrices

The procedure is:

1. Store raw IMU measurements.
2. Convert quaternion to ZYX Euler angles.
3. Remove orientation offset.
4. Convert local angular velocity into the representation required by OCS2.
5. Update the angular state.

1.7.1 Quaternion to Euler Conversion

The orientation is represented as

$$q = \begin{bmatrix} w & x & y & z \end{bmatrix}^T. \tag{1.7}$$

The helper function

`quatToZyx(q)`

returns

$$\boldsymbol{\theta} = \begin{bmatrix} \psi \\ \theta \\ \phi \end{bmatrix}, \tag{1.8}$$

where

- ψ: yaw
- θ: pitch
- ϕ: roll

1.8 Angular State Update

The function

```
updateAngular(
    const vector3_t& zyx,
    const vector_t& angularVel);
```

stores

$$\texttt{rbdState}(0:2) = \texttt{zyx}, \tag{1.9}$$

$$\texttt{rbdState}(N:N+2) = \texttt{angularVel}. \tag{1.10}$$

Therefore, the state contains both orientation and angular velocity.

1.9 Linear State Update

The function

```
updateLinear(
    const vector_t& pos,
    const vector_t& linearVel);
```

stores

$$\texttt{rbdState}(3:5) = \texttt{pos}, \tag{1.11}$$

$$\texttt{rbdState}(N+3:N+5) = \texttt{linearVel}. \tag{1.12}$$

These correspond to the floating-base position and velocity.

1.10 State Estimation Interface

The function

```
virtual vector_t update(
    const rclcpp::Time& time,
    const rclcpp::Duration& period) = 0;
```

is pure virtual and must be implemented by derived classes.
A typical implementation performs:

1. Read IMU measurements.

2. Read joint encoder measurements.
3. Read contact information.
4. Estimate base position and velocity.
5. Update the state vector.
6. Publish odometry.

1.11 Odometry Publishing

The function

```
publishMsgs(const nav_msgs::msg::Odometry& odom);
```

publishes

- `/odom`
- `/pose`

at a fixed rate of approximately

$$f = 200 \text{ Hz}. \tag{1.13}$$

Realtime ROS publishers are used to ensure thread-safe communication.

1.12 Class Architecture

The overall data flow is illustrated in Figure **??**.

In summary, `StateEstimateBase` provides a common interface for sensor handling, state storage, and ROS communication, while derived classes implement the actual state estimation algorithm.

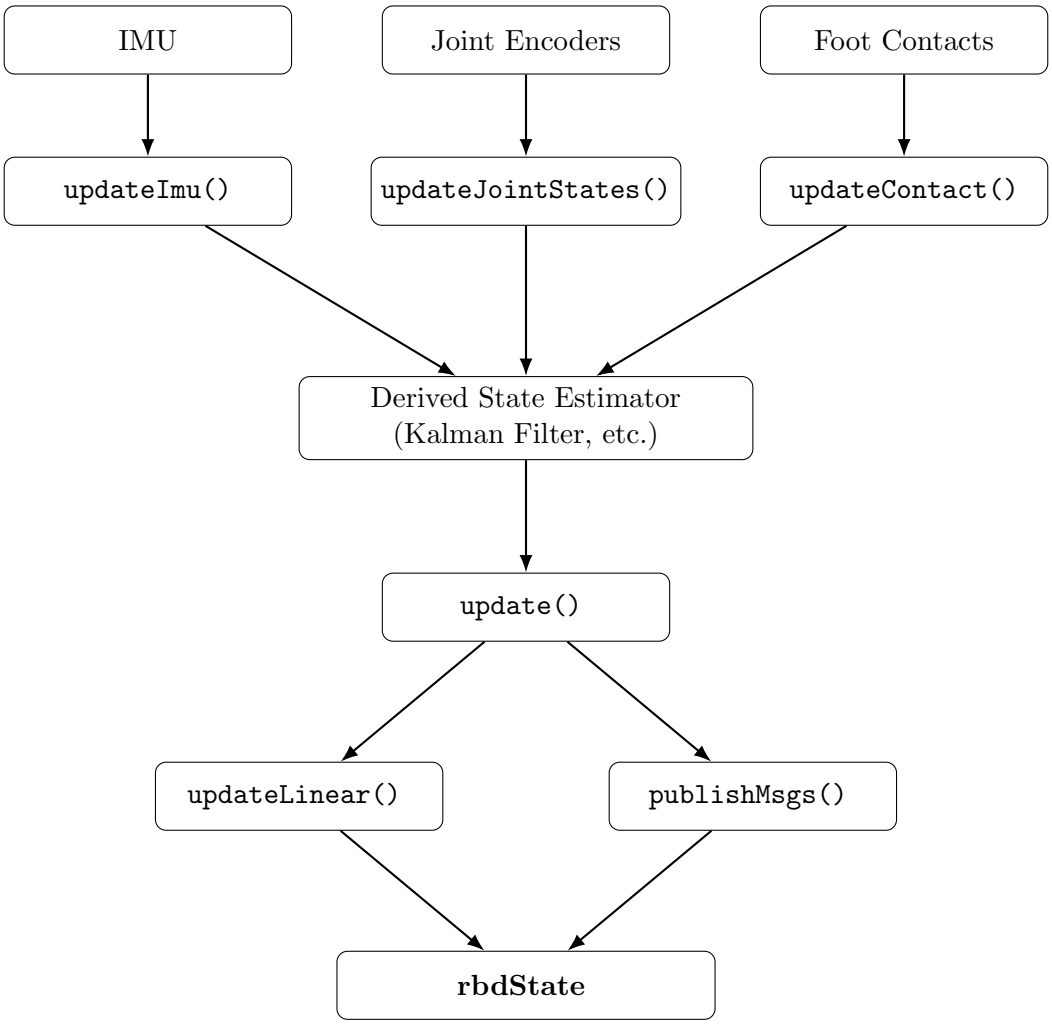


Figure 1: Architecture of the `StateEstimateBase` framework.

2 Linear Kalman Filter State Estimation

The `KalmanFilterEstimate` class implements a discrete-time linear Kalman filter for estimating the floating-base position and velocity of a legged robot. The filter fuses information from:

- IMU linear acceleration,
- Joint encoder measurements,
- Foot kinematics,
- Foot contact states.

2.1 State Vector

Assume the robot has (n_c) contact points. The state vector is defined as

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_b & \mathbf{v}_b & \mathbf{p}_{f_1} & \dots & \mathbf{p}_{f_{n_c}} \end{bmatrix} \in \mathbb{R}^{6+3n_c}, \tag{2.1}$$

where

- ($\mathbf{p}_b \in \mathbb{R}^3$) is the base cartesian position,
- ($\mathbf{v}_b \in \mathbb{R}^3$) is the base cartesian/linear velocity,
- ($\mathbf{p}_{f_i} \in \mathbb{R}^3$) is the world-frame position of foot (i).

Therefore,

$$n_x = 6 + 3n_c. \tag{2.2}$$

2.2 System Dynamics

The state propagation model is

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{a}_k + \mathbf{w}_k, \tag{2.3}$$

where

$$\mathbf{a}_k = \mathbf{R}_{WB}\mathbf{a}_{imu} + \mathbf{g} \tag{2.4}$$

is the IMU acceleration expressed in the world frame.
The state transition matrix is

$$\mathbf{A} = \begin{bmatrix} \mathbf{I}_3 & \Delta t\mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{3n_c} \end{bmatrix}, \tag{2.5}$$

which corresponds to

$$\mathbf{p}_{b,k+1} = \mathbf{p}_{b,k} + \Delta t\mathbf{v}_{b,k}, \tag{2.6}$$
$$\mathbf{v}_{b,k+1} = \mathbf{v}_{b,k}. \tag{2.7}$$

The control-input matrix is

$$\mathbf{B} = \begin{bmatrix} \frac{1}{2}\Delta t^2 \mathbf{I}_3 \\ \Delta t \mathbf{I}_3 \\ \mathbf{0} \end{bmatrix}. \tag{2.8}$$

Hence,

$$\mathbf{p}_{b,k+1} = \mathbf{p}_{b,k} + \Delta t \mathbf{v}_{b,k} + \frac{1}{2}\Delta t^2 \mathbf{a}_k, \tag{2.9}$$

$$\mathbf{v}_{b,k+1} = \mathbf{v}_{b,k} + \Delta t \mathbf{a}_k. \tag{2.10}$$

2.3 Measurement Model

The observation vector is constructed as

$$\mathbf{y} = \begin{bmatrix} \mathbf{p}_s & \mathbf{v}_s & \mathbf{h}_f \end{bmatrix}, \tag{2.11}$$

where

- $(\mathbf{p}_s)$ is the foot position measurement,
- $(\mathbf{v}_s)$ is the foot velocity measurement,
- $(\mathbf{h}_f)$ is the measured foot height.

The code computes

$$\mathbf{p}_s = -\mathbf{p}_{foot}, \tag{2.12}$$

$$\mathbf{v}_s = -\mathbf{v}_{foot}. \tag{2.13}$$

The observation model is

$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{v}, \tag{2.14}$$

with

$$\mathbf{C} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0} & -\mathbf{I} \\ \mathbf{0} & \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{H} \end{bmatrix}. \tag{2.15}$$

and $\mathbf{v} \approx \mathcal{N}(0, \mathbf{R})$ is the measurement noise.
The first block corresponds to

$$\mathbf{p}_s = \mathbf{p}_b - \mathbf{p}_f. \tag{2.16}$$

The second block corresponds to

$$\mathbf{v}_s = \mathbf{v}_b. \tag{2.17}$$

The final block extracts the vertical foot position

$$h_i = z_{f_i}. \tag{2.18}$$

2.4 Process Noise Covariance

The process noise covariance is

$$\mathbf{Q} = \text{diag}\left(Q_p, Q_v, Q_f\right), \tag{2.19}$$

where

$$Q_p = \frac{\Delta t}{20}\sigma_p^2 \mathbf{I}_3, \tag{2.20}$$

$$Q_v = \frac{9.81\Delta t}{20}\sigma_v^2 \mathbf{I}_3, \tag{2.21}$$

$$Q_f = \Delta t \sigma_f^2 \mathbf{I}_{3n_c}. \tag{2.22}$$

When a foot is not in contact, its process noise is increased by a large factor to reduce confidence in the corresponding state estimate.

2.5 Measurement Noise Covariance

The measurement covariance matrix is

$$\mathbf{R} = \text{diag}\left(R_p, R_v, R_h\right), \tag{2.23}$$

where

$$R_p = \sigma_{fp}^2 \mathbf{I}_3, \tag{2.24}$$

$$R_v = \sigma_{fv}^2 \mathbf{I}_3, \tag{2.25}$$

$$R_h = \sigma_{fh}^2 \mathbf{I}_3. \tag{2.26}$$

For swing feet, the corresponding measurement covariance entries are inflated by a large factor, effectively removing them from the filter update.

2.6 Kalman Filter Prediction

The predicted state estimate is

$$\hat{\mathbf{x}}_k^- = \mathbf{A}\hat{\mathbf{x}}_{k-1} + \mathbf{B}\mathbf{a}_k. \tag{2.27}$$

The predicted covariance is

$$\mathbf{P}_k^- = \mathbf{A}\mathbf{P}_{k-1}\mathbf{A}^T + \mathbf{Q}. \tag{2.28}$$

2.7 Kalman Filter Update

The innovation is

$$\mathbf{e}_k = \mathbf{y}_k - \mathbf{C}\hat{\mathbf{x}}_k^-. \tag{2.29}$$

The innovation covariance is

$$\mathbf{S}_k = \mathbf{C}\mathbf{P}_k^-\mathbf{C}^T + \mathbf{R}. \tag{2.30}$$

The Kalman gain is

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{C}^T \mathbf{S}_k^{-1}. \tag{2.31}$$

The corrected state estimate becomes

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \mathbf{e}_k. \tag{2.32}$$

Finally, the covariance update is

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{C})\, \mathbf{P}_k^-. \tag{2.33}$$

2.8 Estimated Robot State

The estimated floating-base state is extracted as

$$\mathbf{p}_b = \hat{\mathbf{x}}(1:3),\ \mathbf{v}_b = \hat{\mathbf{x}}(4:6). \tag{2.34}$$

These values are passed to

```
updateLinear(
xHat_.segment<3>(0),
xHat_.segment<3>(3));
```

to update the robot state representation and generate odometry messages.